

Examic Evaluation System — Frontend Knowledge Transfer

Audience: New engineers joining the project.
Goal: After reading this document you should be able to navigate, modify, and extend the frontend without needing to ask anyone.

Table of Contents

- 1. [Project Overview](#)
- 2. [Tech Stack](#)
- 3. [Directory Structure](#)
- 4. [Application Bootstrap](#)
- 5. [Authentication System](#)
- 6. [Permission & Access-Control System](#)
- 7. [Redux Store \(State Management\)](#)
- 8. [Routing System](#)
- 9. [API Layer & Services](#)
- 10. [Module: CBT \(Computer-Based Testing\)](#)
- 11. [Module: OSM \(Online Script Marking\)](#)
- 12. [Module: Evaluation Flow](#)
- 13. [Key Shared Components](#)
- 14. [Hooks Reference](#)
- 15. [Context Providers](#)
- 16. [Utilities](#)
- 17. [Environment Variables](#)
- 18. [Data Flow: End-to-End Walkthrough](#)
- 19. [Permission Reference Table](#)
- 20. [Common Patterns & Conventions](#)

1. Project Overview

Examic Evaluation System is a multi-module web application for managing academic evaluations. It serves three distinct user groups across three modules:

Module	Acronym	Purpose
Computer-Based Testing	CBT	Evaluate candidates who sat computer-delivered exams. Faculty can view exams, view per-candidate answer scripts, and grade long-answer / rubric-based questions.
Online Script Marking	OSM	Digital scanning and marking of physical paper-based scripts. Admin manages papers and rosters; evaluators mark assigned scripts; reviewers moderate marks.
Evaluation Flow	EF	End-to-end workflow for term-end and internal assessments. Covers roster creation, synoptic uploads, marking pallets, data entry, and manual script marking.

The system uses **role-based privilege gates** to restrict which users can see which modules and pages. A single user account can hold privileges spanning multiple modules.

2. Tech Stack

Core Framework

Library	Version	Purpose
React	18.3.1	UI framework
React DOM	18.3.1	DOM rendering
Vite	—	Build tool & dev server

State Management

Library	Version	Purpose
@reduxjs/toolkit	2.2.7	Redux store, slices, async thanks

Library	Version	Purpose
react-redux	9.1.2	React bindings for Redux
@tanstack/react-query	5.59.15	Server state / data fetching cache

Routing

Library	Version	Purpose
react-router-dom	6.26.1	Client-side routing

UI Libraries

Library	Purpose
@radix-ui/* (20+ packages)	Accessible headless UI primitives (via shadcn/ui)
@mui/material 6.1.1	Occasional Material UI components
lucide-react 0.428.0	Icon library
framer-motion 12.23.0	Animations
Tailwind CSS	Utility-first styling

Data Fetching & API

Library	Purpose
axios 1.7.7	HTTP client
@tanstack/react-query	Query caching, background refetching
@tanstack/react-table	Headless table management

PDF / Canvas / Annotation

Library	Purpose
react-pdf 9.2.1	Render PDF files in browser
react-konva 18.2.10	Canvas-based drawing (annotations)
react-sketch-canvas 6.2.0	Freehand sketch annotations
react-colorful 5.6.1	Color picker for annotation tools

Notifications

Library	Purpose
react-hot-toast 2.5.2	Toast notifications
react-toastify 10.0.5	Alternative toast library
sonner 2.0.7	Lightweight notification system

Cryptography

Library	Purpose
node-forge 1.3.1	RSA-OAEP + AES-CBC decryption of exam question papers

3. Directory Structure

frontend/ └─ src/	
├─ main.jsx	Entry point – mounts Redux Provider + App
├─ App.jsx / App.scss	Root component + global styles
├─ Layout.jsx	Shell layout: ThemeProvider + Navbar + <Outlet />
├─ PrivilegesUpdater.jsx	Background component that syncs user privileges on navigation
├─ api.js	Axios instance with Bearer-token interceptor
├─ index.css	Global CSS resets
├─ config/	
│ └─ PermissionConfig.js	Single source of truth for all permission strings
├─ context/	
│ └─ ThemeProvider.jsx	Theme context (currently enforces light mode only)
├─ hooks/	
│ └─ usePermissions.jsx	Main permission-checking hook with caching
│ └─ useDebounce.jsx	Generic debounce hook
│ └─ useRBAC_old.jsx	LEGACY – do not use
├─ store/	
│ └─ index.js	Redux store configuration + localStorage persistence
│ └─ slices/	
│ │ └─ authSlice.js	Auth state: user, token, roles, permissions
│ │ └─ questionSlice.js	Decrypted CBT exam questions + answers
│ │ └─ annotationsSlice.js	PDF annotation state (drawings, marks)
│ │ └─ rosterSlice.js	Evaluator's assigned OSM rosters
├─ routes/	
│ └─ AppRouter.jsx	Top-level BrowserRouter + route tree
│ └─ routes.jsx	All route definitions with permission requirements
│ └─ evaluationFlowRoutes.jsx	Evaluation Flow module routes
│ └─ ProtectedRoute.jsx	Guards: must be logged in
│ └─ PrivilegeBasedAccessControl.jsx	Guards: must have required permissions
│ └─ RoleBaseAccessControl.jsx	Alternative role-based guard (less used)
├─ services/	
│ └─ CBT/	
│ │ └─ CBTEXamAPIs.js	Fetch exams, question papers, generate answer scripts
│ │ └─ CBTStudentEvaluationAPIs.js	Fetch candidates, evaluate objectives/long answers
│ └─ OSM/	
│ │ └─ OSMServiceAPIs.js	Fetch papers, evaluations, review assignments, rosters
│ │ └─ studentService.js	Hierarchical student data (filters, lists, details)
│ └─ EvaluationFlow/	
│ │ └─ evaluationFlowService.js	Filter options, synoptics, marking pallets, roster engine
│ │ └─ dataEntryService.js	Data entry tests, marks, IC assessments
├─ utils/	
│ └─ decryptJSON.js	Decrypts exam question papers (RSA + AES)
│ └─ shortcuts.js	Keyboard shortcut restrictions (right-click, hotkeys)
├─ components/	
│ └─ Admin/	
│ │ └─ DataSyncModal.jsx	Super-admin data sync interface
│ └─ Canvas/	PDF viewer + annotation tools (drawing, colors, comments)

├─ Errors/	
│ └─ ErrorPage.jsx	Generic 403/404 error display
├─ GradingPage/	CBT grading UI (marks panel, question panel, layouts)
├─ Login/	
│ └─ LoginPage.jsx	Login form with feature showcase carousel
├─ Navbar/	Top navigation bar + theme toggle
├─ Osm/	OSM-specific helper components
├─ PBT/	Paper-Based Testing grading sub-components
├─ common/	ErrorBoundary, Pagination, StyledBreadcrumb
└─ ui/	shadcn/ui components + ExamicLoader spinner
├─ pages/	
│ └─ SmartLandingPage.jsx	Entry gate: redirects based on user permissions
│ └─ MainDashboard/	Multi-module dashboard (shown when user has both CBT + OSM)
│ └─ CBT/	CBT module pages
│ └─ OSM/	OSM module pages
│ └─ EvaluationFlow/	Evaluation Flow module pages
│ └─ DashboardPage/	PBT (Paper-Based Test) dashboard
│ └─ PaperScanning/	Dual-camera paper scanning page
└─ lib/	
│ └─ utils.js	Tailwind class-merge utility (cn())

4. Application Bootstrap

Startup Sequence

```

index.html
├─ main.jsx
│   └─ <StrictMode>
│       └─ <Provider store={store}>      ← Redux store injected globally
│           └─ <App />
│               └─ <AppRouter />
│                   └─ <PrivilegesUpdater />  ← Syncs permissions on each navigation
│                       └─ <BrowserRouter>
│                           └─ <Layout>      ← ThemeProvider + Navbar wrapper
│                               └─ <ProtectedRoute>  ← Auth guard
│                                   └─ <PrivilegeBasedAccessControl> ← Permission guard
│                                       └─ <PageComponent />

```

main.jsx

The entry point. Wraps the entire app in React's `StrictMode` (catches double-render bugs in dev) and Redux `Provider` (makes the store accessible to all components).

```

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <Provider store={store}>
      <App />
    </Provider>
  </StrictMode>
)

```

Layout.jsx

A persistent shell around all authenticated pages. It wraps content in `ThemeProvider` and renders the `Navbar`. The actual page content is injected via React Router's `<Outlet />`.

```
const Layout = () => (  
  <ThemeProvider>  
    <Navbar>  
      <Outlet />    {/* ← current route's page renders here */}  
    </Navbar>  
  </ThemeProvider>  
)  
);
```

PrivilegesUpdater.jsx

An invisible component rendered inside `AppRouter` . It listens for route changes (`useLocation`) and re-fetches the user's profile from the server on every navigation. This ensures that if an admin updates a user's permissions, the change takes effect on the next page visit without requiring a re-login.

```
On every location change:  
→ dispatch(fetchUserData())  
→ clearPermissionCache()
```

5. Authentication System

Login Flow

```
User submits email + password  
→ dispatch(loginUser(credentials))  
→ POST /api/auth/login  
→ Response: { userToken, user: { roles, privileges, ... } }  
→ Token stored in localStorage["token"]  
→ Auth state stored in localStorage["authState"]  
→ clearPermissionCache()  
→ Navigate to "/"  
→ SmartLandingPage redirects based on permissions
```

Auth State Shape (`store/slices/authSlice.js`)

```
{  
  user: {                // User profile object from server  
    _id: string,  
    full_name: string,  
    email: string,  
    roles: string[],  
    privileges: string[], // ← this array drives the permission system  
    ...  
  },  
  userRoles: string[],    // Alias of user.roles  
  permissions: string[],  // Alias of user.privileges (e.g. ["OSM_FACULTY", "CBT_VIEW_DASHBOARD"])  
  token: string | null,   // JWT token  
  isAuthenticated: boolean,  
  isLoading: boolean,  
  error: string | null,  
}
```

Persistence Strategy

What	Where	When
JWT token	<code>localStorage["token"]</code>	Set on login, removed on logout
Full auth state (user + permissions)	<code>localStorage["authState"]</code>	Saved via <code>store.subscribe()</code> on every state change

On app reload What	authSlice.js loadStateFromLocalStorage() Where	Rehydrates the initial Redux state from localStorage When
-----------------------	---	---

Logout

```
dispatch(logout())
// → Clears localStorage["token"]
// → Resets all auth state to null/[]
// → Calls clearPermissionCache()
```

Token Refresh / Expiry

There is currently **no automatic token refresh**. If the token expires mid-session, the next API call will fail. The `fetchUserData` thunk sets `isAuthenticated = false` and clears state if the server rejects the token, which will cause `ProtectedRoute` to redirect to `/login`.

6. Permission & Access-Control System

This is the most critical system to understand. It controls what every user can see and do.

6.1 PermissionConfig.js — The Source of Truth

`src/config/PermissionConfig.js` defines **every permission string** used across the entire application. Never hardcode permission strings — always reference this config.

```
const PermissionConfig = {
  permissions: {
    // Super-level
    SUPER_ADMIN: "SUPER_ADMIN",
    ADMIN_DATA_SYNC: "ADMIN_DATA_SYNC",

    // OSM Module
    OSM_VIEW_ADMIN_DASHBOARD: "OSM_VIEW_ADMIN_DASHBOARD",
    OSM_VIEW_PAPERS: "OSM_VIEW_PAPERS",
    OSM_EVALUATE: "OSM_EVALUATE",
    OSM_REVIEW: "OSM_REVIEW",
    OSM_MANAGE_ROSTER: "OSM_MANAGE_ROSTER",
    OSM_MANAGE_USERS: "OSM_MANAGE_USERS",
    OSM_MANAGE_CONFIG: "OSM_MANAGE_CONFIG",
    OSM_FACULTY: "OSM_FACULTY",

    // CBT Module
    CBT_VIEW_DASHBOARD: "CBT_VIEW_DASHBOARD",
    CBT_EVALUATE: "CBT_EVALUATE",
  },
  // Grouped for convenience
  groups: {
    OSM_ADMIN: ["OSM_VIEW_ADMIN_DASHBOARD", "OSM_VIEW_PAPERS", ...],
    CBT_ALL: ["CBT_VIEW_DASHBOARD", "CBT_EVALUATE"],
  }
};
```

6.2 usePermissions Hook — The Core Hook

`src/hooks/usePermissions.jsx` provides permission-checking utilities to any component. It reads `state.auth.permissions` from Redux and uses an in-memory **Map-based cache** so that the same permission check is not re-evaluated on every render.

Exported functions / hooks

Export	Type	Description
default usePermissions()	Hook	Returns { hasPermission, hasPermissions, hasAnyPermission, permissions }

usePermission(perm) Export	Hook Type	Returns true/false for a single permission Description
useAllPermissions(perms[])	Hook	Returns <code>true</code> only if user has all listed permissions
useAnyPermission(perms[])	Hook	Returns <code>true</code> if user has any listed permission
clearPermissionCache()	Function	Clears the Map cache — called on login, logout, nav
PermissionGuard	Component	Renders children only if a single permission is held
withPermission(perm)	HOC	Wraps a component with a permission check

SUPER_ADMIN bypass

If the user holds `SUPER_ADMIN`, **all permission checks return `true`** automatically. This is baked into the hook logic. SUPER_ADMINs can access every page and feature.

Cache behaviour

First call to `hasPermission("OSM_FACULTY")`:

- reads `state.auth.permissions`
- checks if `SUPER_ADMIN` is present → short-circuit `true`
- else checks if `"OSM_FACULTY"` is in permissions array
- stores result in `permissionCache` Map
- returns result

Subsequent calls with same key:

- returns cached result immediately (no array scan)

Cache is cleared on:

- Login (`loginUser.fulfilled`)
- Logout (logout action)
- Every navigation (`PrivilegesUpdater`)
- Whenever permissions change (`PrivilegesUpdater useEffect`)

Usage examples

```
// 1. Single permission check
const { hasPermission } = usePermissions();
if (hasPermission(PermissionConfig.permissions.OSM_EVALUATE)) { ... }

// 2. Any-of check
const { hasAnyPermission } = usePermissions();
const canSeeAdminSection = hasAnyPermission([
  PermissionConfig.permissions.OSM_VIEW_ADMIN_DASHBOARD,
  PermissionConfig.permissions.SUPER_ADMIN,
]);

// 3. All-of check
const { hasPermissions } = usePermissions();
const canDoEverything = hasPermissions(["OSM_EVALUATE", "OSM_REVIEW"]);

// 4. Get the permissions object (for use in JSX)
const { permissions } = usePermissions();
// permissions === PermissionConfig.permissions (the full map of strings)
```

6.3 PrivilegeGate — Conditional UI Rendering

`src/pages/EvaluationFlow/components/PrivilegeGate.jsx` is a component that hides/shows children based on permissions. Use this inside any page to show a section only to authorised users.

```

<PrivilegeGate
  requiredPermissions={[permissions.OSM_VIEW_ADMIN_DASHBOARD, permissions.SUPER_ADMIN]}
  mode="any"           // "any" (default) = at least one; "all" = must have every one
  fallback=<p>Not authorised</p> // optional – rendered if no access
>
  <AdminSection />
</PrivilegeGate>

```

6.4 PrivilegeBasedAccessControl — Route-Level Guard

`src/routes/PrivilegeBasedAccessControl.jsx` is a route-level guard. It is placed as a wrapper `<Route>` element in `AppRouter`. If the user lacks the required permissions, it redirects to `/unauthorized`.

```

// In AppRouter:
<Route element=<PrivilegeBasedAccessControl requiredPermissions={[ "CBT_VIEW_DASHBOARD" ]} />>
  <Route path="/cbt/dashboard" element=<CBTDashboard /> />
</Route>

```

It uses `useAllPermissions(requiredPermissions)` – so all listed permissions must be held. For empty arrays (`[]`), the check always passes.

6.5 ProtectedRoute — Authentication Guard

`src/routes/ProtectedRoute.jsx` is the outermost route guard. It checks `isAuthenticated` from Redux. If the user is not logged in, it redirects to `/login`.

```

// Pseudocode – sits above PrivilegeBasedAccessControl in the route tree
isAuthenticated ? <Outlet /> : <Navigate to="/login" />

```

6.6 SmartLandingPage — Entry-Point Router

`src/pages/SmartLandingPage.jsx` is what `/` maps to. It reads the user's permissions and routes them to the right module automatically:

```

User lands on "/"
├─ Has OSM + CBT access → Show MainDashboard (module picker)
├─ Has OSM only         → Navigate to /evaluation/dashboard
├─ Has CBT only         → Navigate to /cbt/dashboard
└─ Has neither          → Navigate to /unauthorized

```

6.7 Full Access-Control Decision Tree

```

Request for a protected route
    |
    ▼
ProtectedRoute
isAuthenticated?
  No → /login
  Yes ↓
    |
    ▼
PrivilegeBasedAccessControl
hasAllPermissions(RequiredPermissions)?
  No → /unauthorized
  Yes ↓
    |
    ▼
Page renders
(PrivilegeGate hides/shows sections within the page)

```

7. Redux Store (State Management)

Store Configuration (store/index.js)

The store has four reducers. Redux DevTools are enabled **only in development** (`import.meta.env.DEV`). On every state change, the auth slice is persisted to `localStorage["authState"]` (excluding `isLoading` and `error` — those are transient).

```
store
├─ auth      (authSlice)
├─ questions (questionSlice)
├─ annotations (annotationsSlice)
└─ rosters   (rosterSlice)
```

7.1 authSlice (store/slices/authSlice.js)

Purpose: Manages authentication state — login, logout, user profile, permissions.

State shape:

```
{
  user: object | null,
  userRoles: string[],
  permissions: string[], // drives the entire permission system
  token: string | null,
  isAuthenticated: boolean,
  isLoading: boolean,
  error: string | null,
}
```

Async Thunks:

Thunk	HTTP	Endpoint	When used
<code>loginUser(credentials)</code>	POST	<code>/api/auth/login</code>	On login form submit
<code>fetchUserData()</code>	GET	<code>/api/auth/user</code>	On every navigation (PrivilegesUpdater)

Synchronous Actions:

Action	Effect
<code>logout()</code>	Clears token, resets all state, calls <code>clearPermissionCache()</code>
<code>clearError()</code>	Sets <code>error: null</code>

State Transitions (loginUser):

```
pending    → isLoading: true, error: null
fulfilled  → isAuthenticated: true, user set, permissions set, token saved to localStorage
rejected   → isLoading: false, error: "message from server"
```

State Transitions (fetchUserData):

```
pending    → isLoading: true
fulfilled  → user, userRoles, permissions updated silently
rejected   → isAuthenticated: false, user/token cleared, localStorage["authState"] removed
```

7.2 questionSlice (store/slices/questionSlice.js)

Purpose: Stores decrypted CBT exam questions and candidate answers in memory during a grading session.

State shape:

```
{
  decryptedQuestions: QuestionObject[],
  answers: { [questionId]: answerData },
}
```

Actions:

Action	Payload	Effect
setDecryptedQuestions	{ questions?, answers? }	Merges into state (partial update)
clearDecryptedQuestions	—	Resets to empty arrays/objects
updateQuestionMark	{ questionId, uploadedMarks }	Updates a single question's marks

Note: Questions arrive **encrypted** from the server (RSA+AES). They are decrypted by `utils/decryptJSON.js` and then stored here. This slice is **not persisted** to localStorage — data is re-fetched each session.

7.3 annotationsSlice (`store/slices/annotationsSlice.js`)

Purpose: Tracks PDF annotation state during OSM script marking — drawings, marked questions, and marks.

State shape:

```
{
  imageAnnotations: {
    [questionId]: {
      [imageIndex]: annotationData[]
    }
  },
  markedQuestions: string[], // IDs of questions that have been marked
  marks: MarkObject[],
}
```

Actions:

Action	Payload	Effect
updateAnnotations	{ questionId, imageIndex, annotations }	Sets annotations for a specific image within a question
setMarkedQuestions	string[]	Replaces the entire markedQuestions array
addMarkedQuestion	questionId	Adds a question ID (no duplicates)
removeMarkedQuestion	questionId	Removes a question ID
resetAnnotations	—	Clears everything
updateMarks	MarkObject[]	Replaces marks array

Not persisted — cleared when user navigates away from a grading session.

7.4 rosterSlice (`store/slices/rosterSlice.js`)

Purpose: Fetches and stores the logged-in evaluator's OSM assignment rosters (which scripts they need to evaluate, and which they need to review).

State shape:

```
{
  rosters: EvaluationRoster[],
  status: "idle" | "loading" | "succeeded" | "failed",
  error: string | null,

  reviewRosters: ReviewRoster[],
  reviewStatus: "idle" | "loading" | "succeeded" | "failed",
  reviewError: string | null,
}
```

Async Thunks:

Thunk	HTTP	Endpoint	Description
<code>fetchPrimaryRosters()</code>	POST	<code>/api/osm/evaluation/get-my-evals</code>	Scripts assigned to this evaluator
<code>fetchReviewRosters()</code>	POST	<code>/api/osm/evaluation/get-my-review-evals</code>	Scripts assigned for review/moderation

Both thunks read `user._id` and `token` from the Redux auth state automatically.

8. Routing System

Overview

The app uses React Router v6. All routes are defined in `src/routes/routes.jsx` and `src/routes/evaluationFlowRoutes.jsx`, then assembled in `AppRouter.jsx`.

Route Group Structure

Each entry in `routesList` is a **route group**:

```
{
  id: "group-id",
  RequiredPermissions: ["PERMISSION_STRING"], // checked by PrivilegeBasedAccessControl
  childrens: [                                // actual routes in this group
    {
      id: "route-id",
      path: "/some/path",
      element: <SomePage />,
      breadcrumb: "Display Name",
    }
  ]
}
```

All routes in a group share the same permission gate. A route group with `RequiredPermissions: []` has no gate (accessible to any authenticated user).

Route Map

Evaluation Flow Routes (from `evaluationFlowRoutes.jsx`)

Path	Component	Description
<code>/evaluation/dashboard</code>	<code>EvaluationDashboard</code>	Evaluation Hub home
<code>/evaluation/term-end</code>	<code>TermEndDashboard</code>	Term-end exam scripts
<code>/evaluation/admin</code>	<code>AdminConsole</code>	Admin settings hub
<code>/evaluation/admin/synoptics</code>	<code>SynopticsManager</code>	Upload question paper synoptics
<code>/evaluation/admin/markings-pallet</code>	<code>MarkingPalletManager</code>	Manage marking schemes

Path	Component	Description
/evaluation/admin/roster	RosterManager	Create/manage evaluation rosters
/evaluation/my-evaluations	EvaluatorDashboard	Scripts assigned to me
/evaluation/my-moderations	EvaluatorDashboard	Moderations assigned to me (same component)
/evaluation/manual-marking/:studentUniqueCode	ManualMarkingPage	Mark a specific student script
/evaluation/data-entry	DataEntryPage	Data entry for marks
/evaluation/ic-marks	ICMarksDashboard	Internal continuous assessment marks

OSM Admin Routes (RequiredPermissions: [])

Path	Component	Description
/osm/admin/dashboard	AdminDashboard	OSM admin overview
/osm/eval-roaster	OverallEvaluationRoster	Full evaluation roster view
/osm/papers	PapersDashboard	Manage paper files
/osm/admin/evaluation-dashboard	EvaluatorDashboard	Admin view of evaluations
/osm/admin/reviewer-dashboard	ReviewerDashboard	Admin view of reviews
/osm/admin/results-publication	ResultPublicationPage	Publish results
/osm/students	StudentManagementPage	Student data management

OSM Non-Admin Routes (RequiredPermissions: [])

Path	Component	Description
/osm/dashboard	OSMDashboard	Evaluator home
/osm/evaluation-dashboard	EvaluatorDashboard	My evaluations
/osm/reviewer-dashboard	ReviewerDashboard	My reviews

CBT Routes (RequiredPermissions: ["CBT_VIEW_DASHBOARD"])

Path	Component	Description
/cbt/dashboard	CBTDashboard	Exam list
/cbt/:examCode/:examId/candidates	CBTStudentPage	Candidates for an exam
/cbt/:examCode/:examId/:scheduleId/:candidateId/rubrics-evaluation	RubricsEval	Rubric grading
/cbt/:examCode/:scheduleId/:candidateId/rubrics-evaluation	RubricsEval	Legacy rubric grading URL

Common Routes (RequiredPermissions: [])

Path	Component	Description
/	SmartLandingPage	Permission-based entry point
/pbt/dashboard	DashboardPage	PBT dashboard

Path	Component	Description
/grading	GradingPage	Generic grading page
/osm/grading/:uniqueScriptCode	OSMGradingPage	OSM script grading
/grading-2	GradingPage2	Alternative grading page
/osm/evaluate-script	OSMScreen	Script evaluation screen
/osm/scan-paper	DualCameraScanner	Dual-camera paper scanner

Special Routes

Path	Component	Notes
/login	LoginPage	Outside the auth guard
/unauthorized	ErrorPage	Accessible to authenticated users with wrong permissions
/forbidden	ErrorPage	403 display
*	ErrorPage	404 catch-all

How the Nested Route Guards Work

```
<BrowserRouter>
  <Layout>                                ← Navbar wraps everything
    <ProtectedRoute isAuth={isAuthenticated}> ← Must be logged in
      <PrivilegeBasedAccessControl          ← Must have permissions
        requiredPermissions={['CBT_VIEW_DASHBOARD']}
      >
        <Route path="/cbt/dashboard" element={<CBTDashboard />} />
      </PrivilegeBasedAccessControl>
    </ProtectedRoute>
  </Layout>
</BrowserRouter>
```

9. API Layer & Services

9.1 Axios Instance (src/api.js)

All HTTP calls go through a single shared Axios instance. It auto-attaches the JWT token to every request:

```
const api = axios.create({ baseUrl: import.meta.env.VITE_API_BASE_URL });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

Important: There is no **response interceptor**. If the server returns 401, the request fails silently. The `PrivilegesUpdater` indirectly catches this via `fetchUserData` rejecting and clearing auth state.

9.2 CBT Services (services/CBT/)

CBTExamAPIs.js

Function	Method	Endpoint	Purpose
fetchExams({ orgId })	GET	/cbt/test/get-all-tests	List all CBT exams for an org

Function	Method	Endpoint	Purpose
<code>fetchQuestionsByExamId({ examId, orgId })</code>	GET	<code>/cvt/test/get-question-papers</code>	Fetch encrypted question papers
<code>generateAnswerScriptsAPI({ testCode, testId, orgId })</code>	POST	<code>/cvt/test/generate-answer-scripts</code>	Generate answer scripts for a test
<code>generateAnswerScriptsForScheduleAPI({ ... })</code>	POST	<code>/cvt/test/generate-answer-scripts-for-schedule</code>	Generate scripts for a scheduled test

CBTStudentEvaluationAPIs.js

Function	Method	Endpoint	Purpose
<code>fetchCandidates({ testId, testCode, orgId })</code>	GET	<code>/cvt/test/get-candidates-by-test</code>	List candidates for a CBT test
<code>evaluateObjectives({ ... })</code>	POST	<code>/cvt/objective-evaluation/bulk</code>	Submit objective question marks
<code>evaluateObjectivesForSchedule({ ... })</code>	POST	<code>/cvt/objective-evaluation-for-schedule/bulk</code>	Objective marks for a schedule
<code>fetchAnswerScriptAPI({ candidateId, testCode, orgId })</code>	GET	<code>/cvt/test/get-answer-script</code>	Fetch a candidate's answer script
<code>evaluateLongAnswer({ candidateId, testId, testCode, orgId })</code>	POST	<code>/cvt/candidate/long-answer-evaluation</code>	Submit long-answer evaluation

9.3 OSM Services (`services/OSM/`)

OSMServiceAPIs.js

Function	Method	Endpoint	Purpose
<code>fetchPapers()</code>	GET	<code>/api/osm/get-all-papers</code>	All OSM paper files
<code>fetchEvaluationByFaculty(facultyId)</code>	GET	<code>/api/osm/get-evaluation-paper-by-faculty</code>	Papers assigned to a faculty
<code>fetchAllEvaluations()</code>	GET	<code>/api/osm/get-all-evaluations</code>	Admin: all evaluations
<code>fetchReviewEvaluationByFaculty(facultyId)</code>	GET	<code>/api/osm/get-review-eval-by-faculty</code>	Review papers for a faculty
<code>fetchAllReviewEvaluations()</code>	GET	<code>/api/osm/get-all-review-eval</code>	Admin: all review evaluations
<code>fetchEvalRoster()</code>	GET	<code>/api/osm/get-evaluation-roster</code>	Full evaluation roster

studentService.js

Function	Method	Endpoint	Purpose
<code>fetchFilterOptions(orgId)</code>	GET	<code>/api/osm/students/filters</code>	Filter dropdown options
<code>fetchHierarchicalData(orgId, filters)</code>	POST	<code>/api/osm/students/hierarchical</code>	Tree-structured student data
<code>fetchStudentList(orgId, params)</code>	GET	<code>/api/osm/students/list</code>	Flat student list
<code>fetchStudentDetails(studentId, orgId)</code>	GET	<code>/api/osm/students/:studentId/details</code>	Single student details

9.4 Evaluation Flow Services (`services/EvaluationFlow/`)

evaluationFlowService.js

Function	Purpose
<code>fetchFilterOptions(orgId)</code>	Filter dropdowns (programs, courses, etc.)
<code>fetchAdminCoursesWithStatus(...)</code>	Course list with evaluation status for admin
<code>uploadSynoptic(...)</code>	Upload question paper synoptic PDF
<code>generateSASUrl(...)</code>	Get Azure SAS URL for PDF preview
<code>fetchMarkingPallets(...)</code>	List marking schemes for a course
<code>createMarkingPallet(...)</code>	Create new marking scheme
<code>updateMarkingPallet(...)</code>	Update existing marking scheme
<code>deleteMarkingPallet(...)</code>	Remove a marking scheme
<code>fetchHierarchicalRosterData(...)</code>	Roster data for admin roster builder
<code>fetchRosterByProgram/Exam/Course/Evaluator(...)</code>	Stepped roster creation data
<code>createRoster(...)</code>	Submit roster creation
<code>fetchModerationRuleSets(...)</code>	Moderation rules for automation
<code>fetchEvaluatorAssignments(evaluatorId)</code>	Scripts assigned to an evaluator
<code>fetchScriptsForAssessment(...)</code>	Scripts for a specific assessment
<code>fetchOrgSettings(orgId)</code>	Organisation settings (OSM feature flag, etc.)
<code>fetchCBTStudentSchedules(...)</code>	CBT schedule data for evaluation

dataEntryService.js

Function	Method	Endpoint	Purpose
<code>fetchDataEntryTests({ userId, orgId, purpose })</code>	GET	<code>/api/evaluation/data-entry/tests</code>	Tests available for data entry
<code>fetchStudentsForDataEntry({ examId, courseId, ... })</code>	GET	<code>/api/evaluation/data-entry/students</code>	Student list for a test
<code>saveDataEntryMarks(data)</code>	POST	<code>/api/evaluation/data-entry/save-marks</code>	Save marks for one student
<code>saveDataEntryMarksBulk(data)</code>	POST	<code>/api/evaluation/data-entry/save-marks-bulk</code>	Bulk save marks
<code>fetchICAssessments({ userId, orgId })</code>	GET	<code>/api/evaluation/data-entry/ic-assessments</code>	IC assessments for a faculty

10. Module: CBT (Computer-Based Testing)

Purpose

Faculty evaluate candidates who completed computer-based exams. The module handles: browsing exams → selecting a candidate → viewing their answer script → submitting objective/long-answer marks.

Page Flow

```
/cbt/dashboard (CBTDashboard)
└─ [click exam] → /cbt/:examCode/:examId/candidates (CBTStudentPage)
    └─ [click candidate] → /cbt/:examCode/:examId/:scheduleId/:candidateId/rubrics-evaluation (RubricsEval)
```

Key Pages

CBTDashboard (pages/CBT/CBTDashboard/index.jsx)

- Fetches all CBT exams via React Query (fetchExams)
- Supports **grid and list view** toggle
- Search + filter functionality
- Decrypts question papers using `decryptJSON` utility
- Each exam card shows: name, code, date, status badge, question count
- Click on exam → navigate to candidates page

CBTStudentPage (pages/CBT/CBTStudentPage/index.jsx)

- URL params: `:examCode` , `:examId`
- Fetches candidates for the exam via `fetchCandidates`
- Shows online/offline tabs — `OfflineTabContent` for offline mode
- Click on candidate → navigate to rubrics evaluation with candidate ID + schedule ID

RubricsEval (pages/CBT/RubricsEval/index.jsx)

- URL params: `:examCode` , `:examId` , `:scheduleId` , `:candidateId`
- Fetches candidate answer script
- Renders marking interface for rubric-based questions
- Submits long-answer evaluation marks

GradingPage (CBT grading — pages/CBT/GradingPage/index.jsx)

- Alternative grading interface (referenced as `/grading-2`)
- Uses `CBTLayout` / `PBTLayout` components from `components/GradingPage/`

Data Security

Exam question papers are encrypted server-side. The `decryptJSON.js` utility:

1. Reads private/public keys from env vars (`VITE_TEST_DATA_PRIVATE_KEY` , `VITE_TEST_DATA_PUBLIC_KEY`)
2. Decrypts an RSA-OAEP-wrapped AES key
3. Decrypts the payload with AES-CBC
4. Verifies a Merkle root signature
5. Returns parsed JSON questions

The decrypted questions are stored in the `questions` Redux slice until the session ends.

11. Module: OSM (Online Script Marking)

Purpose

Physical exam papers are scanned and uploaded. Evaluators mark the scanned images digitally. Reviewers moderate the marks. Admins oversee the entire workflow.

User Roles in OSM

Role	Permissions	Can Do
OSM Admin	<code>OSM_VIEW_ADMIN_DASHBOARD</code> , <code>OSM_VIEW_PAPERS</code> , <code>OSM_MANAGE_ROSTER</code> , <code>OSM_MANAGE_USERS</code> , <code>OSM_MANAGE_CONFIG</code>	See all evaluations, manage rosters, view papers, publish results
Evaluator	<code>OSM_EVALUATE</code> , <code>OSM_FACULTY</code>	See assigned scripts, mark them
Reviewer	<code>OSM_REVIEW</code>	Review and moderate marked scripts

Page Flow


```
/osm/dashboard (OSMDashboard)           ← Evaluator home
/osm/admin/dashboard (AdminDashboard) ← Admin home
├─ /osm/papers (PapersDashboard)
├─ /osm/eval-roaster (OverallEvaluationRoster)
└─ /osm/admin/evaluation-dashboard
    └─ /osm/grading/:uniqueScriptCode (OSMGradingPage)
        └─ [open script] → /osm/evaluate-script (OSMScreen)
```

Key Pages

OSMDashboard (pages/OSM/OSMDashboard.jsx)

- Evaluator-facing home page
- Stat cards: Total Scripts, Evaluated Today, Pending Review, Completion Rate
- Navigation cards to Evaluation Dashboard and Reviewer Dashboard
- Animated with Framer Motion

AdminDashboard (pages/OSM/AdminDashboard.jsx)

- Admin overview of the entire OSM system
- System-wide stats, evaluation status overview

EvaluatorDashboard (pages/OSM/EvaluatorDashboard.jsx)

- Lists rosters assigned to the evaluator (from rosterSlice)
- Shows evaluation progress per paper
- Click roster → go to grading page

ReviewerDashboard (pages/OSM/ReviewerDashboard.jsx)

- Lists rosters assigned for review
- Similar structure to EvaluatorDashboard

OSMGradingPage (pages/OSM/OSMGradingPage.jsx)

- Accessed via /osm/grading/:uniqueScriptCode
- Shows a specific script's scanned images
- Integrates PDFCanvas / DraggablePdfViewer for annotation
- Marks panel shows question marks, annotations stored in annotationsSlice

OSMScreen (pages/OSM/OSMScreen.jsx)

- Full-screen script evaluation interface
- Dual-pane: script images on left, marking panel on right

PapersDashboard (pages/OSM/PapersDashboard.jsx)

- Admin page listing all uploaded paper files
- Upload, view, manage papers

StudentManagementPage (pages/OSM/StudentManagementPage.jsx)

- Browse students with hierarchical filters (program → year → course)
- View individual student details

ResultPublicationPage (pages/OSM/ResultPublicationPage.jsx)

- Admin page for publishing final results

PDF Annotation System

The components/Canvas/ folder contains the annotation tools used in OSM marking:

Component	Purpose
PDFCanvas.jsx	Renders PDF pages as canvas elements
DraggablePdfViewer.jsx	PDF viewer with drag-and-drop repositioning
AnnotationContextMenu.jsx	Right-click menu for annotation options

Component	Purpose
<code>ColorPalette.jsx</code>	Provides color picker for drawing tools
<code>NumberedCommentsSidebar.jsx</code>	Sidebar listing numbered comments on a script

Annotations are stored in `annotationsSlice.imageAnnotations[questionId][imageIndex]` .

12. Module: Evaluation Flow

Purpose

The comprehensive workflow engine for term-end exams and internal assessments. It covers the full lifecycle: admin setup → roster creation → faculty marking → data entry → moderation.

User Roles in Evaluation Flow

Who	Permission	Access
Admin / HOD	<code>OSM_VIEW_ADMIN_DASHBOARD</code> , <code>OSM_VIEW_PAPERS</code>	Admin Console, Roster Manager
Faculty / Evaluator	<code>OSM_FACULTY</code>	My Evaluations, IC Marks, Term End
Super Admin	<code>SUPER_ADMIN</code>	Everything

Page Flow

```
/evaluation/dashboard (EvaluationDashboard) ← Hub
├─ Admin Section (needs OSM_VIEW_ADMIN_DASHBOARD or SUPER_ADMIN)
│   ├── /evaluation/admin (AdminConsole)
│   │   ├── /evaluation/admin/synoptics (SynopticsManager)
│   │   └── /evaluation/admin/marketing-pallet (MarketingPalletManager)
│   └── /evaluation/admin/roster (RosterManager)
└─ My Work Section (needs OSM_FACULTY or SUPER_ADMIN)
    ├── /evaluation/ic-marks (ICMarksDashboard)
    └── /evaluation/term-end (TermEndDashboard)
        └── /evaluation/my-evaluations (EvaluatorDashboard)
            └── /evaluation/manual-marking/:studentUniqueCode (ManualMarkingPage)
```

Key Pages

EvaluationDashboard (`pages/EvaluationFlow/Dashboard.jsx`)

- The hub page. Uses `PrivilegeGate` to show/hide **Administration** and **My Work** sections.
- Stats row (currently static – TODO: connect to real API)
- Module cards use Framer Motion animations
- `ModuleCard` sub-component handles hover animation and routing

AdminConsole (`pages/EvaluationFlow/AdminModules/AdminConsole.jsx`)

- Tabbed interface for managing synoptics and marking pallets
- Entry point for all admin configuration tasks

SynopticsManager (`pages/EvaluationFlow/AdminModules/SynopticsManager.jsx`)

- Upload question paper synoptic PDFs to Azure Blob Storage
- Generates SAS URLs for preview
- Course filter → upload → preview workflow

MarkingPalletManager (`pages/EvaluationFlow/AdminModules/MarkingPalletManager.jsx`)

- CRUD interface for marking schemes
- Each pallet defines mark allocations per question

- Used during manual marking

RosterManager (pages/EvaluationFlow/AdminModules/RosterManager.jsx)

- Admin creates evaluation assignments
- Step-by-step: select Program → select Exam → select Course → select Evaluators
- Contains RosterCreationEngine sub-component

EvaluatorDashboard (pages/EvaluationFlow/EvaluatorFlow/EvaluatorDashboard.jsx)

- Shows scripts assigned to the logged-in faculty
- Used for **both** /evaluation/my-evaluations and /evaluation/my-moderations (same component, different mode via URL)

ManualMarkingPage (pages/EvaluationFlow/EvaluatorFlow/ManualMarkingPage.jsx)

- Full marking interface for a specific student's script
- URL param: :studentUniqueCode
- Shows the scanned script + marking pallet side by side

DataEntryPage (pages/EvaluationFlow/EvaluatorFlow/DataEntryPage.jsx)

- For entering marks from physical sheets
- Fetches data entry tests, lists students, saves marks
- Supports both single-save and bulk-save

ICMarksDashboard (pages/EvaluationFlow/EvaluatorFlow/ICMarksDashboard.jsx)

- Internal Continuous Assessment marks entry
- Faculty views their assigned courses and enters IC marks

TermEndDashboard (pages/EvaluationFlow/TermEndDashboard.jsx)

- Lists term-end exam scripts assigned to the faculty

13. Key Shared Components

LoginPage (components/Login/LoginPage.jsx)

- Left panel: rotating feature showcase with auto-play (CBT evaluation, AI grading, digital scripts, analytics)
- Right panel: email/password form with show/hide password toggle
- Dark mode responsive (adapts styling to system preference)
- On submit: dispatch(loginUser({ email, password })) → on success: navigate("/")
- Statistics bar: 50K+ evaluations, 200+ institutions

Navbar (components/Navbar/Navbar.jsx)

- Persistent top navigation
- Shows module links based on user permissions
- User avatar / profile menu with logout
- Breadcrumb integration

StyledBreadcrumb (components/common/StyledBreadcrumb.jsx)

- Renders breadcrumb navigation
- Props: items[], showHome, maxItems, showBackButton
- Used at the top of all major pages

ErrorPage (components/Errors/ErrorPage.jsx)

- Generic 403 / 404 display
- Renders different messages based on the error type

ErrorBoundary (components/common/ErrorBoundary.jsx)

- React error boundary to catch component-level crashes
- Shows a fallback UI instead of crashing the whole app

ExamicLoader (components/ui/ExamicLoader/)

- Branded loading spinner
- Used across the app for async loading states

MainDashboard (pages/MainDashboard/index.jsx)

- Shown to users with access to **both** CBT and OSM
- Module cards for CBT, OSM, Admin Data Sync (SUPER_ADMIN only)
- System stats via React Query (/api/dashboard/overview-stats)
- Recent activity timeline
- Animated with Framer Motion

DataSyncModal (components/Admin/DataSyncModal.jsx)

- Only visible to SUPER_ADMIN users
- Triggers data synchronisation operations

DualCameraScanner (pages/PaperScanning/DualScanner.jsx)

- Uses two camera feeds to scan physical exam papers
- Route: /osm/scan-paper

14. Hooks Reference

usePermissions (hooks/usePermissions.jsx) 📌 Most Important

Returns the full permission utility object. Always import this for permission checks inside components.

```
import usePermissions from "@hooks/usePermissions";

const MyComponent = () => {
  const { hasPermission, hasPermissions, hasAnyPermission, permissions } = usePermissions();
  // permissions === PermissionConfig.permissions (the string map)
};
```

Return value	Type	Description
hasPermission(perm)	(string) => boolean	Check one permission
hasPermissions(perms[])	(string[]) => boolean	True if user has ALL
hasAnyPermission(perms[])	(string[]) => boolean	True if user has ANY
permissions	object	The full <code>PermissionConfig.permissions</code> map

Individual hooks (when you need a hook that returns just a boolean):

```
import { usePermission, useAllPermissions, useAnyPermission } from "@hooks/usePermissions";

const canEvaluate = usePermission("OSM_EVALUATE");
const canDoAll = useAllPermissions(["OSM_EVALUATE", "OSM_REVIEW"]);
const canDoAny = useAnyPermission(["OSM_FACULTY", "SUPER_ADMIN"]);
```

useDebounce (hooks/useDebounce.jsx)

Standard debounce hook for search inputs.

```
import useDebounce from "@hooks/useDebounce";

const [searchTerm, setSearchTerm] = useState("");
const debouncedSearch = useDebounce(searchTerm, 300); // 300ms delay

useEffect(() => {
  // fires only after user stops typing for 300ms
  doSearch(debouncedSearch);
}, [debouncedSearch]);
```

`useRBAC_old` (`hooks/useRBAC_old.jsx`)

DEPRECATED. Do not use. Use `usePermissions` instead.

15. Context Providers

`ThemeProvider` (`context/ThemeProvider.jsx`)

Wraps all authenticated pages (via `Layout.jsx`). Provides theme context but **currently enforces light mode only**. The dark mode toggle in the Navbar is present but non-functional by design.

```
// What the context provides:
{
  themePreference: "light",      // always "light"
  isDarkMode: false,           // always false
  toggleTheme: () => {},        // no-op
  setTheme: (theme) => {},      // no-op
}
```

`document.documentElement.classList` always has the `dark` class removed, and `localStorage` always stores `"light"`. If dark mode needs to be re-enabled, this is where to change it.

16. Utilities

`decryptJSON` (`utils/decryptJSON.js`)

Decrypts exam question papers. Called in CBT flows after fetching encrypted question data from the server.

Algorithm:

1. Receives an encrypted payload from the server
2. Decrypts the AES key using RSA-OAEP (private key from `VITE_TEST_DATA_PRIVATE_KEY`)
3. Decrypts the payload data using AES-CBC
4. Verifies a Merkle root digital signature using `VITE_TEST_DATA_PUBLIC_KEY`
5. Returns parsed JSON

Usage:

```
import { decryptJSON } from "@/utils/decryptJSON";

const questions = await decryptJSON(encryptedPayload);
// questions = [{ id, text, marks, ... }]
```

If decryption fails (wrong key, tampered data), it throws an error — the UI should catch this and show an error state.

`shortcuts` (`utils/shortcuts.js`)

Controls keyboard shortcuts and right-click behaviour during exam sessions.

```
import { handleKeyDown, handleContextMenu, forbiddenShortcuts } from "@/utils/shortcuts";

// Attach to the grading container:
<div onKeyDown={handleKeyDown} onContextMenu={handleContextMenu}>
```

Currently, `forbiddenShortcuts` array is **commented out** (all shortcuts are allowed). The structure is in place for future restriction during sensitive evaluation sessions.

`lib/utils.js` — `cn()` utility

```
import { cn } from "@/lib/utils";

// Merges Tailwind classes safely (handles conditionals and conflicts)
<div className={cn("base-class", condition && "conditional-class", "override-class")} />
```

17. Environment Variables

All env vars are prefixed `VITE_` (required by Vite to expose them to the browser bundle).

Variable	Purpose	Required
<code>VITE_API_BASE_URL</code>	Base URL for all API calls (e.g., <code>https://api.examic.com</code>)	Yes
<code>VITE_TEST_DATA_PRIVATE_KEY</code>	RSA private key for decrypting CBT question papers	Yes (CBT)
<code>VITE_TEST_DATA_PUBLIC_KEY</code>	RSA public key for verifying digital signature	Yes (CBT)

These are stored in `.env` / `.env.local` files (never commit these to git). Copy `.env.example` to get started.

18. Data Flow: End-to-End Walkthrough

Scenario: Faculty logs in and marks an OSM script

```
1. USER opens /login
  └─ LoginPage renders

2. USER submits email + password
  └─ dispatch(loginUser({ email, password }))
  └─ POST /api/auth/login
  └─ Response: { userToken, user: { privileges: ["OSM_FACULTY"], ... } }
  └─ authSlice stores user + token in Redux + localStorage
  └─ clearPermissionCache()

3. APP navigates to "/"
  └─ SmartLandingPage checks:
    canAccessOSM = hasAnyPermission(["OSM_FACULTY"]) → true
    canAccessCBT = hasAnyPermission(["CBT_VIEW_DASHBOARD"]) → false
    → <Navigate to="/evaluation/dashboard" />

4. BROWSER loads /evaluation/dashboard
  └─ PrivilegesUpdater fires fetchUserData() (re-syncs privileges)
  └─ EvaluationDashboard renders
  └─ PrivilegeGate checks permissions:
    - Admin section: user has no OSM_VIEW_ADMIN_DASHBOARD → hidden
    - My Work section: user has OSM_FACULTY → shown
  └─ User sees "Internal Assessments" and "Term End Exams" cards

5. USER clicks "Term End Exams"
  └─ navigate("/evaluation/term-end")
  └─ TermEndDashboard loads
  └─ Shows scripts assigned to this faculty (from evaluationFlowService)

6. USER selects a script
  └─ navigate("/evaluation/manual-marking/STU-ABC-123")
  └─ ManualMarkingPage loads
  └─ Fetches student script PDF + marking pallet
  └─ PDF rendered via react-pdf / react-konva
  └─ Annotations stored in annotationsSlice as user draws

7. USER submits marks
  └─ API call to save evaluation
  └─ Success toast shown
  └─ Navigate back to evaluator dashboard
```

19. Permission Reference Table

Permission String	Module	Who Holds It	What It Unlocks
<code>SUPER_ADMIN</code>	System	Super admin only	All routes, all features, all API calls
<code>ADMIN_DATA_SYNC</code>	System	Admin	DataSyncModal access
<code>OSM_VIEW_ADMIN_DASHBOARD</code>	OSM	OSM Admin	Admin Dashboard, Admin Console
<code>OSM_VIEW_PAPERS</code>	OSM	OSM Admin	Papers Dashboard, SynopticsManager
<code>OSM_EVALUATE</code>	OSM	Evaluator	Script marking
<code>OSM_REVIEW</code>	OSM	Reviewer	Script review/moderation
<code>OSM_MANAGE_ROSTER</code>	OSM	OSM Admin	RosterManager
<code>OSM_MANAGE_USERS</code>	OSM	OSM Admin	StudentManagement
<code>OSM_MANAGE_CONFIG</code>	OSM	OSM Admin	System config
<code>OSM_FACULTY</code>	OSM / EF	Faculty	My Evaluations, IC Marks, Term End, Manual Marking
<code>CBT_VIEW_DASHBOARD</code>	CBT	CBT faculty	CBT Dashboard, Candidates page, RubricsEval
<code>CBT_EVALUATE</code>	CBT	CBT faculty	Submitting CBT evaluations

Key rules:

- `SUPER_ADMIN` bypasses **every** permission check
- `OSM_FACULTY` is the base permission for all Evaluation Flow faculty features
- `OSM_VIEW_ADMIN_DASHBOARD` is the base permission for all admin features in OSM/Evaluation Flow
- Routes with `RequiredPermissions: []` are accessible to **any authenticated user**

20. Common Patterns & Conventions

Import Aliases

The `@` alias maps to `src/`. Always use it instead of relative paths:

```
import usePermissions from "@/hooks/usePermissions"; // Good
import usePermissions from "../../hooks/usePermissions"; // Avoid
```

Data Fetching Pattern

Server data uses React Query; persistent/shared state uses Redux.

```
// Server data (React Query) – exams, papers, rosters
const { data, isLoading, error } = useQuery({
  queryKey: ["exams", orgId],
  queryFn: () => fetchExams({ orgId }),
});

// Shared session state (Redux) – auth, annotations, decrypted questions
const user = useSelector((state) => state.auth.user);
dispatch(updateAnnotations({ questionId, imageIndex, annotations }));
```

Permission Check Pattern

```
// In a component:
const { hasAnyPermission, permissions } = usePermissions();
const isAdmin = hasAnyPermission([permissions.OSM_VIEW_ADMIN_DASHBOARD, permissions.SUPER_ADMIN]);

// In JSX:
<PrivilegeGate requiredPermissions={[permissions.OSM_FACULTY]} mode="any">
  <SomeAdminFeature />
</PrivilegeGate>
```

Animation Pattern (Framer Motion)

Dashboard pages use a standard stagger animation pattern:

```
const containerVariants = {
  hidden: { opacity: 0 },
  visible: { opacity: 1, transition: { staggerChildren: 0.08 } },
};
const itemVariants = {
  hidden: { opacity: 0, y: 20 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.4, ease: "easeOut" } },
};

<motion.div variants={containerVariants} initial="hidden" animate="visible">
  <motion.div variants={itemVariants}>...</motion.div>
</motion.div>
```

UI Components

shadcn/ui components live in `src/components/ui/`. Import them as:

```
import { Button } from "@/components/ui/button";
import { Dialog } from "@/components/ui/dialog";
```

Toasts / Notifications

Use `react-hot-toast` for most notifications:

```
import toast from "react-hot-toast";
toast.success("Marks saved successfully");
toast.error("Failed to load scripts");
```

Error Handling in Async Thunks

All thunks use `rejectWithValue` to pass error messages through Redux:

```
catch (error) {
  return rejectWithValue(error.response?.data?.message || "Fallback message");
}
```

Read `state.auth.error`, `state.rosters.error`, etc. in your components to display these.

File Naming Conventions

Type	Convention	Example
Pages	PascalCase or <code>index.jsx</code> in a folder	<code>CBTDashboard/index.jsx</code>
Components	PascalCase. <code>jsx</code>	<code>StyledBreadcrumb.jsx</code>
Hooks	camelCase starting with <code>use</code>	<code>usePermissions.jsx</code>

Services Type	camelCase + APIs OR Service Convention	CBTExamAPIs.js, evaluationFlowService.js Example
Slices	camelCase + Slice	authSlice.js
Config	PascalCase + Config	PermissionConfig.js

Last updated: May 2026. For questions, reach out to the frontend lead or check the project's GitHub issues.